Requirement 2 Plan — Inventory Service in Go

Nov 2, 2026 · @Ali Asaad

Scope

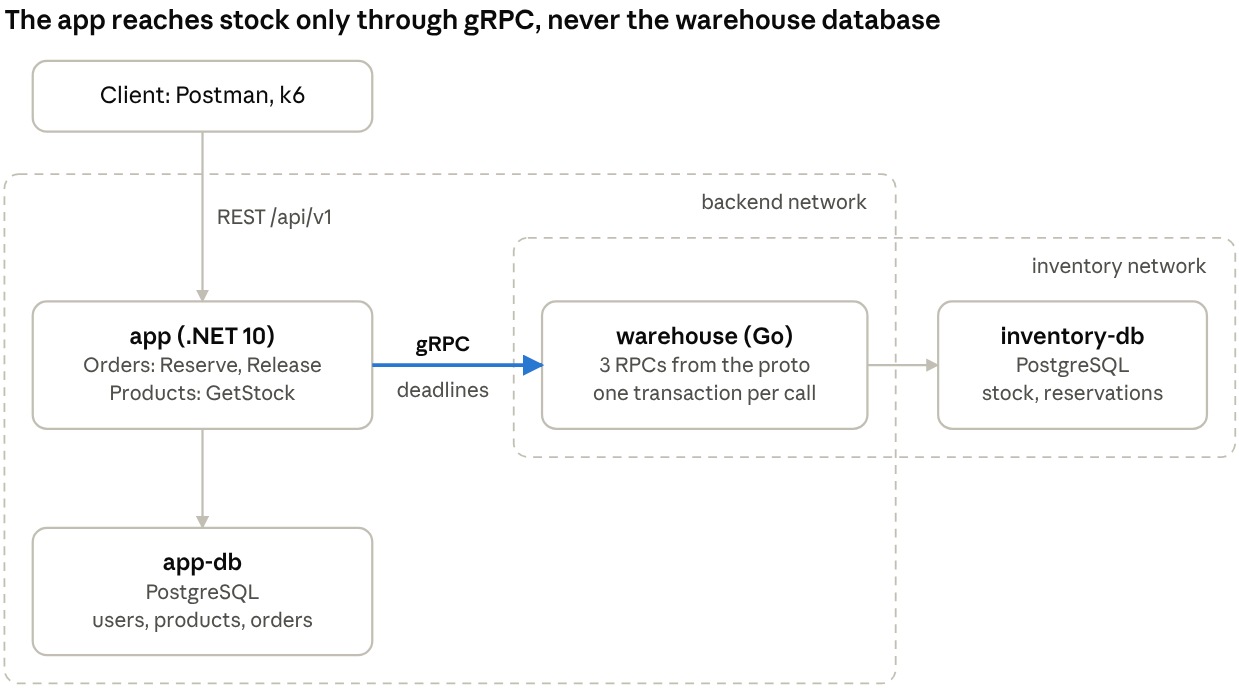
Requirement 2 moves inventory out of the .NET app into a Go service with its own PostgreSQL database, reached only over gRPC through one shared proto contract.

What the brief asks for:

- **A separate service in another language**, with its own database. The app never reads that database; it only calls the contract.
- **Three operations**: get a product's available quantity, reserve the quantities of a whole order in one request, and release a reservation.
- **Order creation** makes one reserve call for all the order's products. Accepted means the order is created; rejected means no order.
- **Product details** show the available quantity, asked from the warehouse.
- **Failure cases**: if saving fails after a reserve, the stock goes back. If the warehouse is down or slow, every call has a deadline and the user gets a clear response. If stock is short, the order is rejected with a clear message.
- **The proof**: several simultaneous orders for the last unit, and exactly one succeeds. The team also shows what the user sees when the warehouse is stopped.

Requirement 1 already routes orders through `IInventoryService` with a placeholder that always reserves, so on the .NET side this is mostly an adapter swap.

Architecture



Requirement 2 architecture · 2 services, 2 databases, 2 networks

The app reaches the warehouse only over gRPC, and `inventory-db` sits on a network the app isn't on, so the app has no route to it. The warehouse listens on port 50051 inside Docker and isn't published to the host.

Stack

Go with the official gRPC and pgx libraries and no web framework: three RPCs need generated types, deadlines and transactions, not routing.

Need	Choice	Why
Language	Go, current stable release, pinned in `go.mod` and the Dockerfile	A different language from the app, as the brief requires; small enough to learn for one service
gRPC server	`google.golang.org/grpc`	One of gRPC's three official core implementations; the caller's deadline arrives as the handler's `context.Context`

Need	Choice	Why
Code generation	Buf CLI: `buf generate`, `buf lint`, `buf breaking`	Generates the Go code from the shared proto; `buf breaking` in CI stops a change that would break the .NET client
Database driver	`github.com/jackc/pgx/v5` with `pgxpool`	Takes the call's context, so a cancelled call cancels its query; `pool.Begin` keeps a transaction on one connection
Health	`google.golang.org/grpc/health`	The standard gRPC health service, for compose and the app's health check
Logs	`log/slog` with JSON output	Structured logs carrying `order_id`, from the standard library
Config	Environment variables	Same pattern as the app: secrets come from `.env` through compose
Tests	`go test -race` and testcontainers-go (Postgres module)	Real Postgres for the race test; the race detector also checks the Go code
Schema	Plain SQL files, run by the Postgres image on first start	Three small tables don't need a migration tool; the tests load the same files
Image	Multi-stage build to a distroless, non-root image	Small image with no shell; a `healthcheck` subcommand in the binary replaces curl

Contract

One proto file in `/contracts/inventory/v1/` is all the two services share; merge it first so both sides can build against it in parallel.

```
syntax = "proto3";

package inventory.v1;

// The only way into stock. order_id makes Reserve and Release safe to
repeat.
service InventoryService {
  // How many units of one product are available.
  rpc GetStock(GetStockRequest) returns (GetStockResponse);
  // Reserves every line of one order, or none of them.
  rpc Reserve(ReserveRequest) returns (ReserveResponse);
  // Gives back what an order reserved. Safe to call twice.
  rpc Release(ReleaseRequest) returns (ReleaseResponse);
}

message GetStockRequest {
  string product_id = 1; // UUID
}

message GetStockResponse {
  int32 available = 1;
}

message ReserveItem {
  string product_id = 1; // UUID
  int32 quantity = 2;    // greater than 0
}

message ReserveRequest {
  string order_id = 1;   // UUID, created by the app before the call
  repeated ReserveItem items = 2;
}

message Shortage {
  string product_id = 1;
  int32 requested = 2;
  int32 available = 3;
}

message ReserveResponse {
  bool reserved = 1;
  repeated Shortage shortages = 2; // the short products, when reserved is
false
```

```
}

message ReleaseRequest {
  string order_id = 1;
}

message ReleaseResponse {}
```

Rules both sides follow:

- `order_id` is the idempotency key: the same Reserve twice reserves once, and Release is safe to repeat.
- Short stock is a normal answer (`reserved = false` with shortages). gRPC errors are only for real failures: `INVALID_ARGUMENT`, `UNAVAILABLE`, `DEADLINE_EXCEEDED`, `INTERNAL`.
- An unknown product counts as 0 available.
- Changes are additive: new fields take new numbers, and existing numbers are never reused. CI runs `buf breaking` against `main`.
- Go code comes from `buf generate` (managed mode sets `go_package`); C# comes from Grpc.Tools with `GrpcServices="Client"`.

Warehouse design

Correctness lives in PostgreSQL: one transaction per call, a conditional update per item and one row per order make the last-unit race and retries safe.

```
CREATE TABLE stock_levels (
    product_id uuid    PRIMARY KEY,
    available  integer NOT NULL CHECK (available >= 0)
);

-- One row per order: 'reserved', or 'released' once given back (a
tombstone).
CREATE TABLE reservations (
    order_id   uuid        PRIMARY KEY,
    status     text        NOT NULL CHECK (status IN ('reserved',
'released')),
    updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE reservation_lines (
    order_id   uuid    NOT NULL REFERENCES reservations (order_id),
    product_id uuid    NOT NULL REFERENCES stock_levels (product_id),
    quantity   integer NOT NULL CHECK (quantity > 0),
    PRIMARY KEY (order_id, product_id)
);
```

Reserve, in one transaction on the call's context:

1. Insert `(order_id, 'reserved')` with `ON CONFLICT DO NOTHING`. If the row already exists, answer from its status: `reserved` means a retry (reply reserved), `released` means the order was given up (reply not reserved).
2. Merge duplicate products and sort by product ID, so two orders lock rows in the same order and can't deadlock.
3. For each item, `UPDATE stock_levels SET available = available - qty WHERE product_id = $1 AND available >= qty`. Zero rows means roll back and reply not reserved, with that item's shortage.
4. Insert the lines, commit, reply reserved.

Release, in one transaction:

1. Insert `(order_id, 'released')` with `ON CONFLICT DO NOTHING`, so a Release that arrives before its Reserve leaves a tombstone.
2. Lock the row with `SELECT … FOR UPDATE`. If it is `reserved`, add each line back in product-ID order and set it to `released`.
3. Commit. An order already released is a no-op.

Why the tombstone: when Reserve times out, the app calls Release, but the original Reserve may still be running. The tombstone makes that late Reserve reply not reserved instead of holding stock for an order that doesn't exist.

Go carries the deadline all the way down: the handler's `ctx` goes into every pgx call, so when the app gives up, the query is cancelled and the transaction rolls back.

```
func (s *Store) Reserve(ctx context.Context, orderID string, items []Item)
(Result, error) {
        tx, err := s.pool.Begin(ctx) // ctx carries the app's deadline
        if err != nil {
                return Result{}, err
        }
        defer tx.Rollback(ctx) // no-op once committed

        tag, err := tx.Exec(ctx, `INSERT INTO reservations (order_id, status)
                VALUES ($1, 'reserved') ON CONFLICT (order_id) DO NOTHING`,
orderID)
        if err != nil {
                return Result{}, err
        }
        if tag.RowsAffected() == 0 {
                return s.answerFromStatus(ctx, tx, orderID) // a retry, or a
tombstone
        }

        for _, it := range mergeAndSort(items) {
                tag, err := tx.Exec(ctx, `UPDATE stock_levels SET available =
available - $2
                        WHERE product_id = $1 AND available >= $2`,
it.ProductID, it.Quantity)
                if err != nil {
                        return Result{}, err
                }
                if tag.RowsAffected() == 0 {
                        return s.shortage(ctx, tx, it) // the deferred
Rollback undoes earlier lines
                }
        }

        // insert reservation_lines here, then:
        if err := tx.Commit(ctx); err != nil {
```

```
                return Result{}, err
        }
        return Result{Reserved: true}, nil
}
```

Seed both databases with the same fixed product IDs, including one product with stock 1 for the demo. Restocking isn't one of the three operations; add a `SetStock` RPC later if the team needs it.

```
warehouse/
├─ cmd/warehouse/main.go     # config, pgx pool, gRPC server, health,
graceful stop
├─ internal/inventory/
|  ├─ server.go              # the three handlers: validation and gRPC status
codes
|  ├─ store.go               # transactions and SQL
|  └─ store_test.go          # race, retry, release and tombstone tests
├─ gen/inventory/v1/         # written by buf generate, never edited by hand
├─ db/
|  ├─ 001_schema.sql         # run by the inventory-db container on first
start
|  └─ 002_seed.sql           # same product IDs as the app's seed
├─ buf.gen.yaml
├─ Dockerfile                # multi-stage, distroless, non-root
└─ go.mod
```

App side (.NET)

One adapter changes: a gRPC client replaces the Requirement 1 placeholder behind `IInventoryService`, so Orders and Products keep their code.

- **Client**: `Grpc.Net.ClientFactory`, `Google.Protobuf` and `Grpc.Tools`, with the proto included as `GrpcServices="Client"`, registered with `AddGrpcClient`.
- **Address**: `Inventory:Address` in config; compose sets `Inventory__Address=http://inventory:50051`, plain HTTP/2 inside the Docker network.
- **Deadlines**: from config, per call. Start with 2 s for Reserve and Release and 500 ms for GetStock, then tune.

- **Retries**: retry `UNAVAILABLE` a few times with backoff through the gRPC client's retry policy. It's safe because all three RPCs can be repeated, and retries stay inside the deadline.
- **Health**: an unreachable warehouse makes the app Degraded, not Unhealthy, so browsing keeps working.

Case	What the app does	Client sees
Not enough stock	No order saved	409 with `shortages`
Warehouse down	No order saved	503 with `Retry-After`
Warehouse slow (deadline passed)	Calls Release in case the reserve landed; no order saved	504
Save fails after a reserve	Calls Release	500
Release itself fails	Writes a `pending_releases` row; a background worker retries it until it succeeds	Same response as the case that triggered it
Admin cancels an order	Calls Release, with the same retry path	200
Product details while the warehouse is down or slow	Returns the product without stock	200 with `stockStatus: "unknown"`

All error responses keep the Requirement 1 Problem Details format and error codes.

Tasks

Twelve PR-sized tasks; merge the contract first, then the Go line and the .NET line run in parallel.

Same four streams as Requirement 1: **A** platform, **B** identity and security, **C** products and inventory, **D** orders. S is about half a day, M 1–2 days, L about 3 days.

#	Task	Who	Size	After	Status
R2-00	Go primer for everyone: the Tour of Go (basics, errors, `context`), then a hello-world gRPC call from .NET to Go in compose	All	S	—	Not started

#	Task	Who	Size	After	Status
R2-01	Contract: `inventory.proto`, `buf lint` and `buf breaking` in CI, Go code via `buf generate`, C# via Grpc.Tools	A	S	—	Not started
R2-02	Warehouse skeleton: env config, pgx pool, gRPC server with health service, `slog` JSON logs, graceful stop on SIGTERM, Dockerfile with a `healthcheck` subcommand	B	M	R2-01	Not started
R2-03	`inventory-db`: schema and seed SQL run on first start, seed IDs matching the app	A	S	R2-02	Not started
R2-04	Store and handlers: GetStock, Reserve and Release as designed above; bad input returns `INVALID_ARGUMENT`	C	L	R2-03	Not started
R2-05	Warehouse tests on real Postgres: 20 goroutines reserve the last unit and exactly one wins; the same `order_id` twice reserves once; Release restores stock; a Reserve after its Release is refused; all under `go test -race`	B	M	R2-04	Not started
R2-06	.NET adapter replacing the placeholder: `AddGrpcClient`, per-call deadlines, retry on `UNAVAILABLE`, `RpcException` mapped to domain errors	D	M	R2-01	Not started
R2-07	App failure handling from the table above, including the `pending_releases` worker and the Degraded health check	D	M	R2-06	Not started
R2-08	`GET /products/{id}` shows available stock from GetStock	C	S	R2-06	Not started
R2-09	Compose: `inventory` and `inventory-db`, `inventory-db` on its own network, secrets from `.env`, `depends_on` with health conditions	A	S	R2-02, R2-03	Not started
R2-10	Proof kit: k6 race script, steps for warehouse stopped and warehouse slow (dev-only `INVENTORY_DELAY_MS`), a .NET test for release on save failure	A	M	R2-07, R2-09	Not started
R2-11	ADR on why Go, contract notes, demo script, and a mock interview where each person walks through the Go code	All	S	All	Not started

The longest chain is the Go line, R2-01 to R2-05; D codes the .NET side against the placeholder until the warehouse is up. Every member reviews at least one Go PR, since anyone can be asked about the warehouse.

Interview proof

Six steps show the race, the down and slow cases, recovery, and the tests.

1. On a clean clone, run `docker compose up`; the seed gives product X a stock of 1.
2. Run the k6 script: one logged-in customer fires 20 simultaneous orders for X, since one login stays under the login rate limit. Expect one 201 and nineteen 409s, then stock 0 on X's page.
3. Run `docker compose stop inventory` and place an order: a 503 Problem Details comes back within the deadline, and X's page still loads with stock unknown.
4. Start the warehouse with `INVENTORY_DELAY_MS` above the deadline: the order gets a 504, and stock is unchanged afterwards.
5. Restart it normally: orders work again, and both services log the same `order_id` for each call.
6. Show `go test -race ./...` and `dotnet test` passing, including the 20-goroutine race test.